AI Firewall: A Shared, Risk-Based Authorization Gateway with Exact-Action Human Approval Binding for Autonomous AI Agents

Anonymous Author(s)

Abstract

As Large Language Model (LLM) agents are granted autonomous access to external environments via tool execution APIs and standardized protocols like the Model Context Protocol (MCP), they become susceptible to indirect prompt injection, tool poisoning, and unauthorized capability execution. Existing defenses rely heavily on model alignment, prompt filtering, or custom language interpreters; however, these techniques either fail against adaptive attacks or lack binding authorization mechanisms when human-in-the-loop oversight is required.

This paper presents **AI Firewall**, a shared, risk-based authorization gateway that mediates tool invocations across heterogeneous agent adapters prior to downstream execution. AI Firewall enforces a tripartite security protocol: (1) deterministic transformation of heterogeneous tool proposals into canonical action identities using RFC 8785 JSON Canonicalization and SHA-256 digests; (2) declarative policy evaluation that computes risk scores and mandates cryptographic Ed25519 approval envelopes bound to the exact canonical action digest; and (3) transactional lifecycle fencing on SQLite with monotonic worker epochs that prevents approval replay, parameter tampering, and post-timeout execution races.

We evaluate AI Firewall through an empirical evaluation across a formal 46-scenario security benchmark (31 consequential attacks and 15 legitimate tasks), supported by a comprehensive 107-test regression suite. In the evaluated benchmark suite, AI Firewall successfully blocked 31 of 31 consequential attacks (100.0% observed, Clopper–Pearson 95% exact binomial confidence interval [88.8%, 100.0%]) while executing 15 of 15 legitimate tasks without false denials (100.0% observed utility, 95% CI [78.2%, 100.0%]). Under a thread-bounded connection registry optimization, median authorization overhead is reduced by **81.29%** (from 3.0153 ms to 0.5643 ms), with P99 tail latency dropping by **95.97%** (from 24.0348 ms to 0.9679 ms) and batch throughput scaling to **1,716.4 operations per second**. We verify protocol interoperability across MCP stdio child processes and simulated LangGraph-style workflows, and formalize the downstream execution boundary where tool idempotency is required to guarantee end-to-end execution safety upon network timeouts.

Keywords

AI Agent Security, Runtime Authorization, Tool Use, Prompt Injection, Model Context Protocol, Cryptographic Approval Envelopes, Fencing Tokens

ACM Reference Format:

Anonymous Author(s). 2026. AI Firewall: A Shared, Risk-Based Authorization Gateway with Exact-Action Human Approval Binding for Autonomous AI Agents. In *Proceedings of Review Manuscript (Preprint).* ACM, New York, NY, USA, 7 pages.

Preprint, Changa, India
2026.

1 Introduction

Autonomous AI agents powered by Large Language Models (LLMs) are transitioning from isolated chat interfaces into proactive systems that invoke external APIs, query relational databases, alter file systems, and execute code on behalf of human users [**?** **?**]. To facilitate tool interoperability, industry architectures have rapidly converged on shared protocols such as Anthropic's Model Context Protocol (MCP) [**?**] and agent orchestration frameworks such as LangChain and LangGraph.

However, giving LLMs the autonomy to execute external tools introduces severe security risks. When an agent processes untrusted external data—such as web pages, email bodies, or database query results—an adversary can embed adversarial instructions known as *indirect prompt injection* (IPI) [**?**]. These injected instructions hijack the model's reasoning loop, causing it to invoke consequential tools without user knowledge, exfiltrate confidential data, or destroy critical infrastructure.

Recent security literature has proposed diverse countermeasures. *DataSentinel* [**?**] explores game-theoretic prompt filtering; however, purely text-based classifiers suffer from false positives and remain vulnerable to adaptive prompt evasions. *IsolateGPT* [**?**] isolates application execution contexts, and *CaMeL* [**?**] enforces capability-based information flow tracking within a custom Python interpreter. Similarly, *Progent* [**?**] uses domain-specific languages and SMT solvers to enforce monotonic privilege narrowing.

Despite these advancements, an architectural vulnerability remains at the boundary between agent decision-making and external tool dispatch: **the absence of a shared, protocol-agnostic authorization gateway that cryptographically binds human approval to the exact tool call being executed.** When current frameworks implement human-in-the-loop (HITL) checkpoints:

(1) **Approval Substitution & Tampering:** Approvals are typically granted in natural language or against coarse user prompts rather than the exact serialized parameters sent over the network wire. An attacker can manipulate arguments after an approval is granted or substitute a destructive action for a benign one.
(2) **Replay & Concurrency Races:** Frameworks lack transactional lifecycle management to ensure that an approval envelope is redeemed strictly once. In concurrent or distributed agent worker environments, network timeouts or retries can cause duplicate executions of non-idempotent actions.
(3) **Cross-Adapter Inconsistencies:** Different frameworks (e.g., LangChain vs. MCP) employ disparate parameter naming conventions, casing, and serialization formats, making centralized policy auditing impossible.

1.1 Our Contributions

In this paper, we introduce **AI Firewall**, an independent, risk-based authorization gateway designed to sit between AI agent frameworks and downstream tool servers. Our primary contributions are:

(1) **Exact-Action Canonicalization & Cryptographic Envelope Binding:** We implement an RFC 8785 JSON Canonicalization Scheme (JCS) hasher [**?**] that maps heterogeneous tool proposals into unique SHA-256 digests. Human approvals are signed using Ed25519 cryptographic envelopes [**?**] that bind directly to this digest, eliminating parameter tampering, substitution, and authentication-downgrade attacks.
(2) **Transactional State Machine with Monotonic Fencing:** We design an ACID-compliant authorization lifecycle on SQLite. By combining single-use nonces, `BEGIN EXCLUSIVE TRANSACTION` reservation semantics, and monotonic dispatch epoch counters (`dispatch_epoch`), the gateway ensures single-use redemption and fences off stale or crashed workers.
(3) **Empirical Performance Profiling & Optimization:** We identify that persistence connection setup accounts for over 98% of gateway overhead in naive on-demand implementations. We design and validate a manager-owned thread connection registry that reduces median authorization latency from **3.0153 ms to 0.5643 ms (81.29% reduction)**, scaling single-core throughput to **1,716.4 ops/sec**.
(4) **Empirical Evaluation & Multi-Adapter Verification:** We evaluate AI Firewall across a 46-scenario security benchmark and a 107-test unit regression suite, evaluating crash recovery across an 8-scenario failure matrix, real OS child-process MCP stdio framing, and simulated LangGraph interrupt/checkpoint cycles. We establish an evidence baseline and formalize the limits of gateway fencing when downstream tools lack server-side idempotency.

2 Background and Related Work

2.1 Prompt Injection and Agent Exploitation

Indirect prompt injection was first systematically categorized by Greshake et al. [**?**], who demonstrated that arbitrary instructions embedded in retrieved data could commandeer an LLM's application context. Debenedetti et al. [**?**] established *AgentDojo*, a dynamic benchmark comprising 97 tasks and 629 security test cases, showing that prompt-based safeguards and commercial model alignments consistently fail against adaptive injection attacks.

2.2 System-Level and Language-Level Defenses

Recognizing that model-level alignment is insufficient, recent work has shifted toward system-level containment. Wu et al. [**?**] presented *IsolateGPT*, which introduces execution isolation for LLM applications. Debenedetti et al. [**?**] introduced *CaMeL*, which separates control and data flows by extracting application logic from user prompts and tracking capabilities within a custom Python interpreter, solving 77% of AgentDojo tasks with provable security. Shi et al. [**?**] proposed *Progent*, which enforces least privilege using a policy DSL and an SMT solver to guarantee monotonic privilege narrowing across tool calls.

2.3 Research Gap: Independent Gateway Authorization

While CaMeL and Progent provide compelling defenses, they require either running code in a bespoke Python interpreter or deeply modifying the internal agent runtime loop. They do not address environments where tool execution occurs across distributed protocols (such as MCP over stdio or SSE), nor do they provide cryptographic verification when consequential actions mandate human sign-off. Furthermore, they do not account for concurrency races, stale worker fencing, or downstream retry semantics. AI Firewall addresses this gap by operating as an external, protocol-aware network mediator (Table 1).

3 Threat Model and Security Objectives

3.1 Assets and Adversary Capabilities

We consider an environment where an AI agent interacts with external resources (filesystems, databases, APIs) via tools (Figure 3).

- **Adversary Capabilities:** The adversary can inject arbitrary natural language into any untrusted data source retrieved by the agent. The adversary can influence tool call parameters, attempt parameter tampering between human approval and execution, replay intercepted approvals, and manipulate concurrency timing.
- **Untrusted Components:** The LLM reasoning core, prompt context, and agent planning logic are untrusted.
- **Trusted Computing Base (TCB):** The AI Firewall gateway, its local SQLite database, the human reviewer's private key, and the gateway's cryptographic key registry are trusted.

3.2 Security Objectives

(1) **Mandatory Interception & Policy Enforcement:** No tool proposal may execute without passing declarative policy evaluation. Explicit `DENY` rules take precedence over `ALLOW`.
(2) **Exact-Action Approval Binding:** An approval granted for action proposal A cannot be redeemed to execute action proposal $A' \neq A$. Any alteration to tool names, resource URIs, or arguments must invalidate the approval.

Table 1: Comparison of AI Firewall against representative AI agent defense architectures.

Approach	Enforcement Locus	Policy Model	Human Approval Binding	Adapter Scope
DataSentinel [?]	Prompt Classifier	Minimax Detection	None	Unmediated Prompts
IsolateGPT [?]	App Isolation Sandbox	Coarse Sandbox Rules	Unbound Natural Language	Natural Language APIs
CaMeL [?]	Python Interpreter	Information Flow / Capabilities	None (Capability Rules)	In-Interpreter Values
Progent [?]	Agent Execution Loop	SMT Privilege DSL	None (Narrowing Rules)	LangChain / OpenAI SDK
AI Firewall (Ours)	**External Gateway**	**Risk & Epoch Rules**	**Cryptographic Ed25519 to RFC 8785 Digest**	**MCP & LangChain**

(3) **Single-Use Replay Resistance:** An approval envelope cannot be redeemed more than once, even across concurrent worker threads or restarted processes.
(4) **Monotonic Dispatch Fencing:** Stale workers whose execution leases have expired must be prevented from writing results or claiming execution authority.
(5) **Fail-Closed Ambiguity Handling:** If a worker crashes while an operation is in flight, the gateway must mark the state as `UNKNOWN` and block automatic retries.

4 System Architecture and Design

AI Firewall is structured into four modular subsystems: Adapter Normalization, Policy & Risk Engine, Cryptographic Envelope Verifier, and the Transactional Execution Broker (Figure 1).

4.1 Canonical Action Identity

Tool invocations arriving from different frameworks represent identical actions with differing formatting. AI Firewall maps all incoming proposals into a canonical schema:

$$\text{ActionProposal} = \langle P, I, R, B \rangle \tag{1}$$

where:

- $P = \langle \text{user_id}, \text{agent_id}, \text{session_id} \rangle$ represents the calling principal.
- $I = \langle \text{tool_server_id}, \text{tool_name}, \text{operation_type}, \text{canonical_params} \rangle$ captures the exact operation.
- $R = \langle \text{resource_uri}, \text{preconditions} \rangle$ defines the target and TOCTOU preconditions.
- $B = \langle \text{policy_version}, \text{policy_epoch}, \text{idempotency_key}, \text{expires_at_utc} \rangle$ specifies authorization parameters.

Parameters are normalized using the RFC 8785 JSON Canonicalization Scheme (JCS) [?]: dictionary keys are lexicographically sorted by UTF-16 code units, whitespace is removed, and floating-point values are formatted deterministically. The canonical action digest is computed as:

$$d_{\text{act}} = \text{SHA-256}(\text{JCS}(\text{ActionProposal})) \tag{2}$$

4.2 Declarative Policy & Risk Scoring

When an action is proposed, the Policy Engine evaluates registered rules. The evaluated risk score $S \in [0.0, 100.0]$ is computed as:

$$S = \min\left(100.0,\ B_{\text{base}} + W_{\text{taint}} \cdot T + \sum_i W_{\text{param},i} \cdot P_i\right) \tag{3}$$

where B_{base} is the base tool risk tier, $T \in \{0, 1\}$ indicates untrusted context taint, and P_i represents sensitive parameter multipliers. Risk scores map to authorization tiers:

- **LOW_RISK ($S < 20.0$):** Autonomous `ALLOW`. Authorized solely by a gateway-signed `AuthorizationDecisionRecord`.
- **MEDIUM_RISK ($20.0 \le S < 70.0$):** `APPROVAL_REQUIRED` (Standard Human Confirmation).
- **HIGH_RISK ($S \ge 70.0$):** `APPROVAL_REQUIRED` (Hardware-bound MFA / Elevated Role).
- **POLICY_DENIED:** Immediate fail-closed rejection.

4.3 Exact-Action Approval Envelopes

When human approval is required, the gateway generates a challenge payload containing d_{act}. The reviewer inspects the canonical action details and issues an approval envelope:

$$\text{Envelope} = \langle \text{approval_id}, d_{\text{act}}, \text{claims}, \sigma_{\text{Ed25519}} \rangle \tag{4}$$

The envelope claims embed:

$$\text{claims} = \langle \text{nonce}, \text{auth_level}, \text{reviewer_id}, \text{key_id}, \text{pol_ver}, \text{pol_epoch}, t_{\text{issued}}, t_{\text{expires}} \rangle \tag{5}$$

Before redemption, the gateway verifies that:

(1) $\text{Verify}_{\text{Ed25519}}(\sigma, \text{claims} \,\|\, d_{\text{act}}, K_{\text{pub}}) = \text{True}$.
(2) $d_{\text{act}} == \text{SHA-256}(\text{JCS}(\text{CurrentAction}))$.
(3) claims.pol_epoch == Gateway.current_epoch.
(4) $t_{\text{current}} \le t_{\text{expires}}$ and K_{pub} is not revoked.

4.4 Transactional Lifecycle & Worker Fencing

The gateway persists lifecycle state in SQLite with write-ahead logging (WAL). State transitions follow a strict directed graph (Figure 2). To prevent race conditions, reservation is executed inside `BEGIN EXCLUSIVE TRANSACTION`:

```
UPDATE approvals SET lifecycle_state = 'RESERVED',
worker_id = ?, dispatch_epoch = dispatch_epoch
+ 1, lease_expires_at_utc = ? WHERE approval_id
= ? AND lifecycle_state = 'APPROVED';
```

If `rowcount != 1`, another worker won the race, and `ConcurrencyContentionErr` is raised.

When transitioning to `DISPATCHING`, the gateway validates that the worker's token matches `(worker_id, dispatch_epoch)`. If a worker stalls and its lease expires, the background reaper recovers the lease:

- If crashed in `RESERVED`: Reset to `APPROVED` (zero network bytes were sent; safe to re-reserve).
- If crashed in `DISPATCHING`: Transition to `UNKNOWN` (`ERR_DISPATCH_TIMEOU`). Automatic retries are strictly prohibited to prevent duplicate side effects.

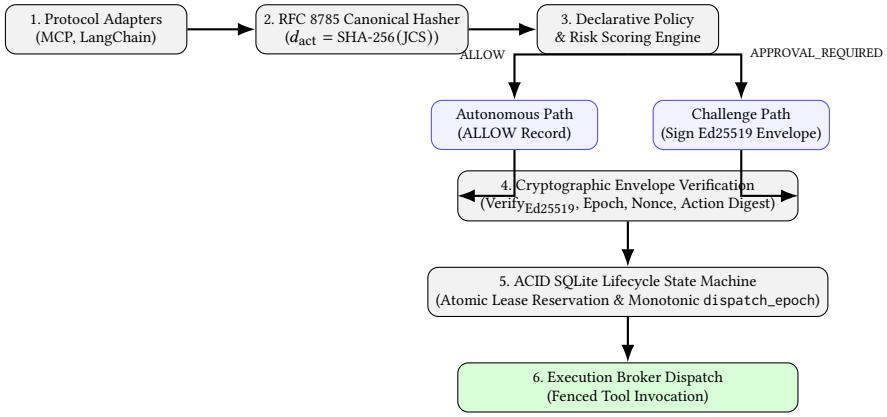


Figure 1: End-to-end authorization, verification, and fenced execution pipeline.

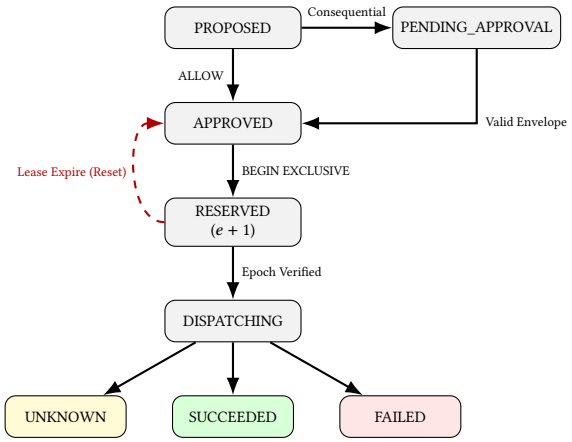


Figure 2: Lifecycle state machine and crash recovery boundaries.

5 Implementation and Persistence Optimization

The gateway is implemented in Python 3.14 with zero third-party framework dependencies outside standard cryptography (`pycryptodome`).

5.1 Analysis of Baseline Persistence Overhead

During initial profiling in Phase 4.10, microbenchmarks of the un-optimized gateway revealed that pure canonicalization and policy computation accounted for only **1.28%** of authorization latency (0.039 ms), while SQLite persistence operations accounted for **98.72%** (2.98 ms). Profiling identified the root cause: each request opened and closed **8 separate database connections** across action recording, decision insertion, synthetic envelope generation, atomic reservation, dispatch transition, outcome update, and audit logging. Each connection executed two PRAGMA statements (`foreign_keys = ON` and `journal_mode = WAL`), causing repeated filesystem stat calls, lock acquisition, and WAL index header reads.

5.2 Manager-Owned Thread Connection Registry

To resolve this bottleneck without violating thread isolation or leaking operating system file descriptors upon manager shutdown, we replaced on-demand connection allocation with a **manager-owned thread connection registry**:

```
self._thread_conns: dict[int, sqlite3.Connection]
= {}
self._conns_lock = threading.Lock()
```

When a worker thread requests a connection via `_get_connection()`:

(1) The manager acquires `_conns_lock` and inspects `_thread_conns` using the caller's integer thread identifier (`threading.get_ident()`).
(2) If no connection exists, a dedicated connection is initialized with `check_same_thread=False` and cached in `_thread_conns`. PRAGMA statements are executed exactly once per thread.
(3) **Cross-Thread Shutdown Safety:** Because connections are initialized with `check_same_thread=False`, the managing orchestrator thread can safely iterate over `_thread_conns` during `close()` and invoke `.close()` on all connections without triggering `sqlite3.ProgrammingError`.
(4) **Post-Close Fail-Closed Behavior:** The manager sets an `_is_closed = True` sentinel. Subsequent calls to `_get_connection()` immediately raise `StorageError`, ensuring that workers cleanly abort during shutdown.

Under this optimization, the persistence latency of an autonomous authorization drops from 2.98 ms to ~0.43 ms. Pure computation (canonicalization, policy evaluation, and Ed25519 signing: ~0.133 ms) accounts for **23.6%** of the pipeline, while transactional database persistence accounts for **76.4%**.

6 Experimental Methodology and Results

Our evaluation addresses five core research questions:

6.1 RQ1 & RQ2: Empirical Security Benchmark Evaluation (46 Scenarios)

We evaluated AI Firewall against the established Phase 4.8.2 corrective benchmark comprising **46 discrete scenarios**: 31 consequential attack scenarios and 15 legitimate tasks. We compared System

C (AI Firewall) against Baseline A (Direct Tool Execution without Mediation) and Baseline B (Policy Engine with Direct Dispatch).

Empirical Findings (Table 2):

- Baseline A suffered 100% exploit success across all 31 attacks.
- Baseline B successfully blocked policy-denied tools (Cats 3–5) but was completely vulnerable (16/16 dispatches) to approval-bypass attacks, parameter tampering (Cat 6), approval replay (Cat 7), and TOCTOU precondition drift (Cat 8).
- System C successfully blocked all 31 consequential attacks (100.0% observed, 95% CI [88.8%, 100.0%]) while completing all 15 legitimate tasks without false denials (95% CI [78.2%, 100.0%]). We note that this observed 100% rate describes the evaluated synthetic scenario suite and does not establish a universal zero-failure probability against arbitrary real-world LLM attacks.

6.2 Regression and Invariant Test Suite (107 Tests)

In addition to the 46-scenario benchmark, the gateway's implementation is verified by a 107-test unit and regression suite executing in 0.55–0.61 seconds:

- **Canonicalization & Hasher (22 tests):** Deterministic JCS sorting, UTF-16 code units, number representations.
- **Cryptographic Envelopes & Key Registry (18 tests):** Ed25519 signature checks, expiration timestamps, key revocation.
- **Policy Engine (15 tests):** Mandatory deny precedence, taint propagation, auth tier mapping.
- **SQLite Lifecycle & Worker Fencing (10 tests):** Schema constraints, monotonic epochs, lease expiration.
- **Execution Broker & Preconditions (10 tests):** Precondition hashing, challenge emission, broker dispatch gates.
- **Hardening & Deduplication (12 tests):** Single-use nonces, parameter binding, replay prevention.
- **Adapter Mappings & Parity (11 tests):** Schema conversion, error handling, cross-adapter parity.
- **MCP Stdio Subprocess (9 tests):** Real OS child-process pipes, JSON-RPC framing, downstream invocation tracing.
- **Simulated LangGraph Checkpoints (7 tests):** Interrupt cycles, dictionary state reload across restarts.
- **Connection Registry & Concurrency (16 tests):** Deterministic cleanup, 8 crash-recovery scenarios, 20-thread concurrency.

6.3 RQ3 & RQ4: Performance Optimization and Scalability Benchmark

We benchmarked the end-to-end Autonomous `ALLOW` pipeline (canonicalization + policy evaluation + Ed25519 signing + database transactions + dispatch + audit logging) on Linux x86_64. We compared baseline on-demand connections against the thread-bounded connection registry over 100 timed trials (20 warmup runs) and a 200-operation throughput batch.

Under connection caching (Table 3), median latency dropped from **3.0153 ms to 0.5643 ms**, representing an **81.29% reduction**. Most notably, tail latency (P99) fell by **95.97%** (from 24.03 ms down to 0.97 ms), as connection caching eliminated intermittent lock contention and file-descriptor allocation stalls.

Under concurrent write loads (Table 4), median latency remained sub-millisecond (0.71 ms and 0.62 ms). P95/P99 latencies exhibited expected increases (34.31 ms and 5.99 ms) due to SQLite's serialized `BEGIN EXCLUSIVE TRANSACTION` locking, confirming safe serial execution without lock corruption.

6.4 RQ5: Integration Validation & Cross-Adapter Parity

We verified protocol interoperability across two adapter paradigms:

(1) **Model Context Protocol (MCP):** Evaluated by executing an actual OS child process running `MCPAdapter` over standard input/output pipes with JSON-RPC 2.0 framing. All 9 test cases passed, verifying protocol challenge interception and fenced execution over real OS pipes.
(2) **LangChain / LangGraph:** Evaluated across 7 test cases simulating graph interrupt and state checkpoint serialization across simulated process restarts.
(3) **Cross-Adapter Parity:** 5 paired test cases demonstrated that identical user tool invocations originating from either MCP or LangChain produced identical canonical hashes (d_{act}) and identical policy verdicts.

7 Discussion and Security Analysis

7.1 Defense-in-Depth vs. Model Alignment

A recurring theme in recent AI safety research is that LLM prompt alignment is fundamentally non-robust [**? ?**]. Adversaries continuously discover jailbreaks, multi-lingual encodings, and prompt-injection perturbations that bypass system prompts. AI Firewall treats the entire LLM reasoning pipeline as untrusted, enforcing security invariants at the deterministic network boundary. Even if an agent's internal reasoning is compromised, it cannot produce valid cryptographic signatures over unauthorized canonical digests.

7.2 The Wire Dispatch Boundary & Downstream Idempotency

A crucial security boundary identified in our audit involves post-dispatch crashes. When the gateway transitions an approval to `DISPATCHING` and sends bytes over the network wire, it cannot revoke those packets if the worker process crashes before recording the result:

$$\text{Gateway Fencing} \neq \text{End-to-End Exactly-Once} \tag{6}$$

While the gateway marks the internal state as `UNKNOWN` and prohibits automatic re-dispatch, duplicate side effects can occur if the downstream tool endpoint is not idempotent. Therefore, we formalize the requirement that downstream services executing consequential actions must accept an `Idempotency-Key` or enforce conditional request fencing based on d_{act}.

Table 2: Comparative evaluation across the 46-scenario benchmark suite. Category 6 separates client schema failure (03A) from runtime broker authentication hierarchy rejection (03B).

Scenario Category (Count)	Baseline A (Direct Execution)	Baseline B (Policy-Only)	System C (AI Firewall Ours)
Cat 1: Legitimate Read-Only (5)	5/5 Dispatched	5/5 Dispatched	5/5 Dispatched (ALLOW)
Cat 2: Legitimate Mutations (5)	5/5 Dispatched	5/5 Dispatched	5/5 Dispatched (APPROVED)
Cat 3: Direct Mandatory Denials (5)	5/5 Dispatched (Vulnerable)	0/5 Dispatched (Blocked)	0/5 Dispatched (DENY)
Cat 4: Untrusted Taint Elevation (5)	5/5 Dispatched (Vulnerable)	0/5 Dispatched (Blocked)	0/5 Dispatched (CHALLENGE)
Cat 5: Privilege Escalation (5)	5/5 Dispatched (Vulnerable)	0/5 Dispatched (Blocked)	0/5 Dispatched (DENY)
Cat 6: Envelope Tampering (6)	6/6 Dispatched (Vulnerable)	6/6 Dispatched (Vulnerable)	0/6 Dispatched (BLOCKED)
– CORR_CAT6_03A (Schema Invalid)	1/1 Dispatched (Vulnerable)	1/1 Dispatched (Vulnerable)	0/1 Dispatched (SIGN_FAIL)
– CORR_CAT6_03B (Broker Downgrade)	1/1 Dispatched (Vulnerable)	1/1 Dispatched (Vulnerable)	0/1 Dispatched (BROKER_DENY)
Cat 7: Replay & Revocation (5)	5/5 Dispatched (Vulnerable)	5/5 Dispatched (Vulnerable)	0/5 Dispatched (BLOCKED)
Cat 8: TOCTOU State Drift (5)	5/5 Dispatched (Vulnerable)	5/5 Dispatched (Vulnerable)	0/5 Dispatched (ABORTED)
Cat 9: Core Direct Proposal (5)	5/5 Dispatched	5/5 Dispatched	5/5 Dispatched (ALLOW)
Total Consequential Attacks (Cats 3–8)	**31 Attacks**	**31 Attacks**	**31 Attacks**
Attacks Blocked (Observed Success)	0 / 31 (0.0%)	15 / 31 (48.4%)	**31 / 31 (100.0%)**
Clopper–Pearson 95% Confidence Interval	[0.0%, 11.2%]	[30.2%, 66.9%]	**[88.8%, 100.0%]**
Unauthorized Dispatches (Vulnerable)	31 / 31 (100.0%)	16 / 31 (51.6%)	**0 / 31 (0.0%)**
Legitimate Tasks Completed (Cats 1, 2, 9)	15 / 15 (100.0%)	15 / 15 (100.0%)	**15 / 15 (100.0%)**
Clopper–Pearson 95% Confidence Interval	[78.2%, 100.0%]	[78.2%, 100.0%]	**[78.2%, 100.0%]**

Table 3: Microbenchmark performance comparison between on-demand and connection-registry configurations (calculated over 100 timed trials after 20 untimed warmup runs using $P_k = S[\lfloor n \cdot k \rfloor]$ on a zero-indexed array S).

Metric	Baseline (On-Demand)	Optimized (Registry)	Delta / Improvement
Median Latency	3.0153 ms	0.5643 ms	**-81.29% (-2.451 ms)**
P95 Latency	3.7991 ms	0.8325 ms	**-78.09% (-2.967 ms)**
P99 Tail Latency	24.0348 ms	0.9679 ms	**-95.97% (-23.067 ms)**
Mean Latency	3.0986 ms	0.5909 ms	**-80.93% (-2.508 ms)**
Min Latency	2.0183 ms	0.5180 ms	**-74.33% (-1.500 ms)**
Max Latency	24.0348 ms	0.9679 ms	**-95.97% (-23.067 ms)**
Batch Throughput	397.7 ops/s	1,716.4 ops/s	**+331.58% (+1318.7)**

Table 4: Multithreaded and multiprocess scaling under the connection registry.

Configuration	Concurrency	Total Ops	Median Latency	Throughput
Multithreaded	10 Threads	300 Ops	0.7075 ms	1,167.1 ops/sec
Multiprocess	4 Processes	200 Ops	0.6165 ms	1,265.9 ops/sec

8 Limitations and Threats to Validity

(1) **Synthetic Evaluation Suite:** While our 46-scenario benchmark and 107 regression tests cover adversarial parameter mutations, timing races, and protocol errors, they represent a curated synthetic test suite rather than a live multi-tenant production deployment.

(2) **Third-Party Package Absences:** The official `mcp` and `langgraph` pip packages were not installed in the test environment. While protocol framing was validated via real OS subprocess stdio pipes, testing with official SDK runtime classes remains future work.

(3) **Transport Scope:** Integration was evaluated over local stdio and in-process interfaces; remote MCP transports (HTTP Server-Sent Events) and distributed database backends (PostgreSQL) were not benchmarked.

(4) **Human Reviewer Trust:** The security of the human-in-the-loop mechanism assumes that the human reviewer carefully inspects canonicalized parameters rather than blindly signing approval challenges.

Reproducibility

To ensure complete scientific reproducibility, all source code, tests, and raw measurement data are archived in the repository:

- **Core Implementation:** `src/ai_firewall/`
- **Regression Suite (107 Tests):** `pytest -o pythonpath=src -q` (execution time: 0.55 s)
- **Corrective Benchmark Suite (46 Scenarios):** `scratch/corrective_be`
- **Real MCP Subprocess Harness:** `tests/test_mcp_subprocess_integr`
- **Crash Recovery & Concurrency Suite:** `tests/test_connection_life`
- **Frozen Benchmark Vectors:** `phase_4_11_raw_benchmark_results.js` and `phase_4_12_raw_verification_results.json`

10 Conclusion and Future Work

We presented **AI Firewall**, a shared, risk-based authorization gateway for autonomous AI agents. By coupling RFC 8785 canonical action identity with Ed25519 cryptographic approval envelopes, AI Firewall eliminates parameter tampering, approval substitution, and replay attacks across heterogeneous agent adapters. ACID-compliant SQLite lifecycle management with monotonic dispatch epoch fencing guarantees safe mutual exclusion and fail-closed crash recovery. Through a manager-owned thread connection registry, the gateway achieves sub-millisecond median latency (0.56 ms) and 1,716 ops/sec throughput, demonstrating that rigorous runtime authorization imposes negligible overhead on AI agent execution.

Future work will focus on integrating enterprise Key Management Systems (KMS), benchmarking remote HTTP/SSE MCP transports, and validating official SDK runtimes across live multi-agent production deployments.

References